PROPOSED ADAM FOR SEGMENTATION

XU Chentianye

Things worth a mention …

The codes are available at

https://github.com/XuChentianye/Adaptive_Dual_Attention_Module

XuChentianye Add files via upload — a29ef18 4 minutes ago — 12 commits

LICENSE	Initial commit	3 days ago
README.md	Add files via upload	4 minutes ago
adamnet.py	Add files via upload	42 minutes ago
dataset.py	Add files via upload	2 days ago
indicators.py	Add files via upload	2 days ago
loss_fn.py	Add files via upload	2 days ago
pre-trained_models.py	Add files via upload	2 days ago
train.py	Add files via upload	24 minutes ago
utils.py	Add files via upload	3 days ago

README.md

Adaptive_Dual_Attention_Module

My implementation of the paper [Wu, Huisi, et al. "Automated skin lesion segmentation via an adaptive dual attention module." IEEE Transactions on Medical Imaging 40.1 (2020): 357-370.]

Project Modules

- `adamnet.py` net architecture definitions including ADAM and several ADAMNets.
- `dataset.py` ISIC-2017 dataset preprocessing and relavent augmentation.
- `indicators.py` model performance indicator definitions including DCS, JSI, and etc.
- `loss_fn.py` DiceCE loss function.
- `pretrained_models.py` URLs of pretrained ResNet models.
- `train.py` ADAMNet instance creation, ADAMNet training and testing.
- `utils.py` ResNet creation helper.

OUTLINE

This presentation is divided into 6 sections:

1. The Architecture
2. Dataset and Dataloader
3. Data Augmentation
4. Training
5. Evaluating Indicator
6. Experiment Results & Analysis

OUTLINE

This presentation is divided into 6 sections:

1. The Architecture ☜
2. Dataset and Dataloader
3. Data Augmentation
4. Training
5. Evaluating Indicator
6. Experiment Results & Analysis

ARCHITECTURE: Main Paper

IEEE TRANSACTIONS ON MEDICAL IMAGING, VOL. 40, NO. 1, JANUARY 2021 357

Automated Skin Lesion Segmentation via an Adaptive Dual Attention Module

Huisi Wu, Junquan Pan, Zhuoying Li, Zhenkun Wen, *Member, IEEE,* and Jing Qin, *Member, IEEE*

***Abstract*—We present a convolutional neural network (CNN) equipped with a novel and efficient adaptive dual attention module (ADAM) for automated skin lesion segmentation from dermoscopic images, which is an essential yet challenging step for the development of a computer-assisted skin disease diagnosis system. The proposed ADAM has three compelling characteristics. First, we integrate two global context modeling mechanisms into the ADAM, one aiming at capturing the boundary continuity of skin lesion by global average pooling while the other dealing with the shape irregularity by pixel-wise correlation. In this regard, our network, thanks to the proposed ADAM, is capable of extracting more comprehensive and discriminative features for recognizing the boundary of skin lesions. Second, the proposed ADAM supports multi-scale resolution fusion, and hence can capture multi-scale features to further improve the segmentation accuracy. Third, as we harness a spatial information weighting method in the proposed network, our method can reduce a lot of redundancies compared with traditional CNNs. The proposed network is implemented based on a dual encoder architecture, which is able to enlarge the receptive field without greatly increasing the network parameters. In addition, we assign different dilation rates to different ADAMs so that it can adaptively capture distinguishing features according to the size of a lesion. We extensively evaluate the proposed method on both ISBI2017 and ISIC2018 datasets and the experimental results demonstrate that, without using network ensemble schemes, our method is capable of achieving better segmentation performance than state-of-the-art deep learning models, particularly those equipped with attention mechanisms.**

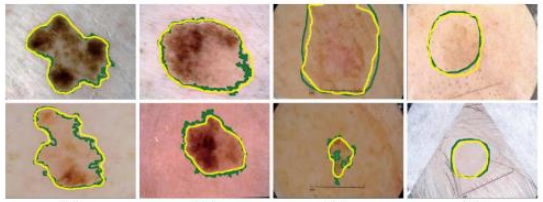


Fig. 1. Typical challenging cases of skin lesion segmentation: (a) cases with irregular shapes, (b) cases where the pixel variations within the lesion are quite large, (c) cases where the pixel variations between the skin and the lesion are small, and (d) the color of the skin and the lesion is different from case to case. The proposed method can well deal with most of these challenging cases, where the green and yellow contours denote the results delineated by dermatologists and our results, respectively.

***Index Terms*—Skin lesion segmentation, adaptive dual attention module, dual encoder architecture, global context modeling, deep learning.**

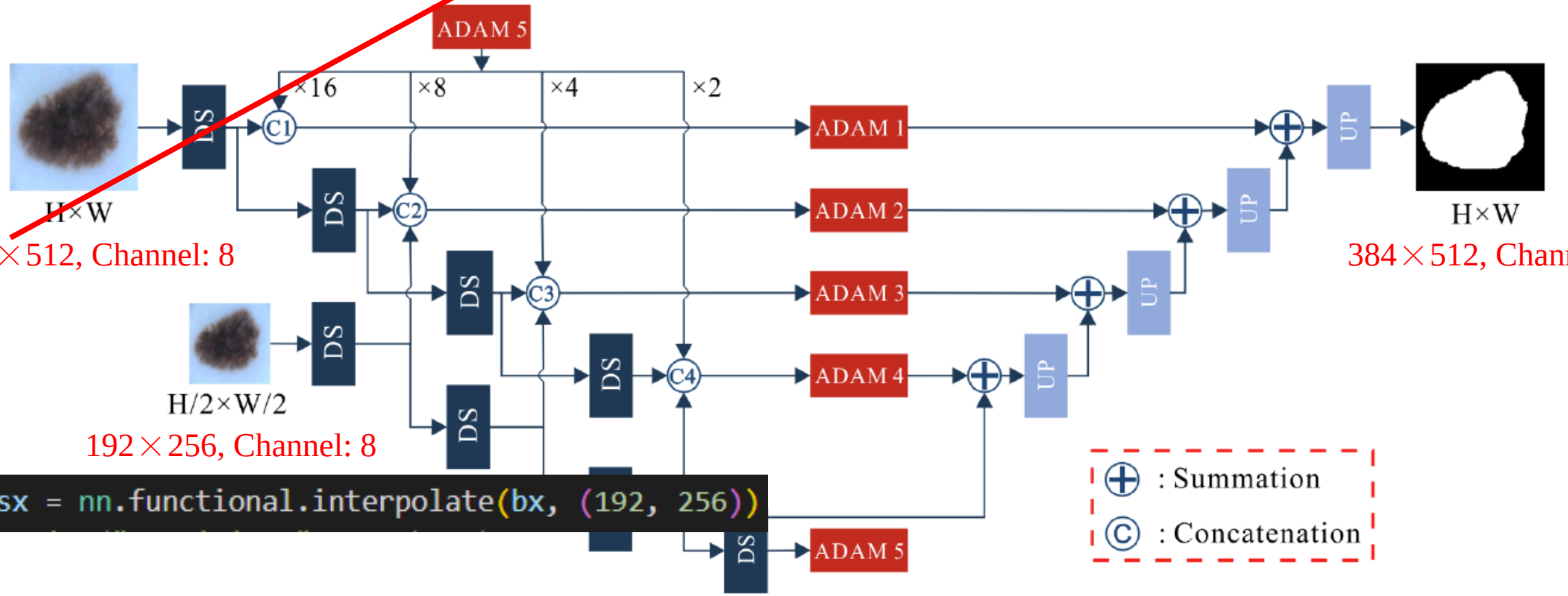
I. Introduction

SKIN cancer is one of the most rapidly increasing cancers all over the world, with estimated 1,329,779 newly diagnosed cases in 2018 [1]. Melanoma is the most deadly skin cancer responsible for an overwhelming majority of skin cancer deaths. The global incidence of melanoma is

bicubic interpolation

ARCHITECTURE: BA

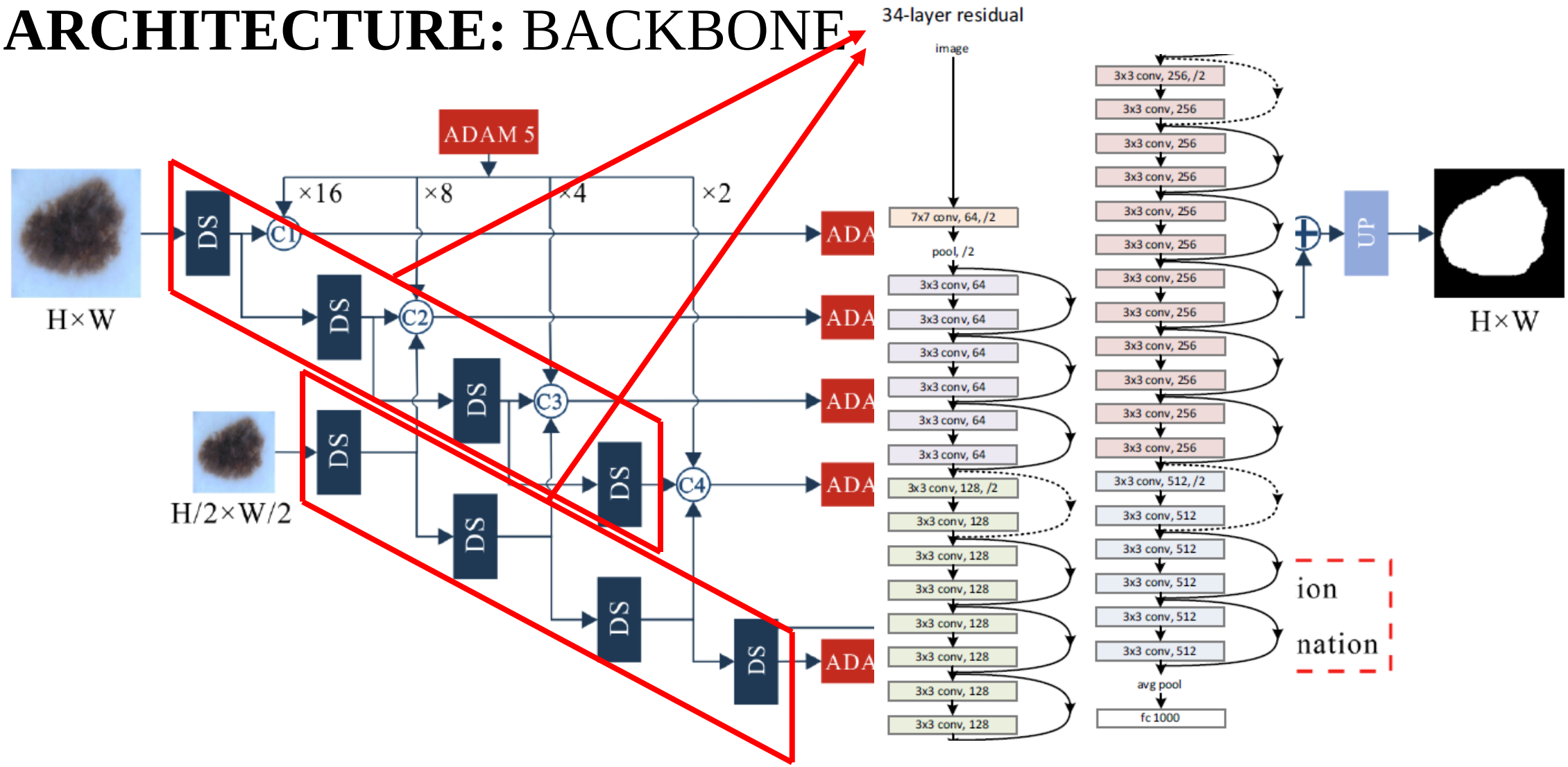
```
self.img_transform = transforms.Compose([
    transforms.Resize((384, 512), interpolation=InterpolationMode.BICUBIC),
```



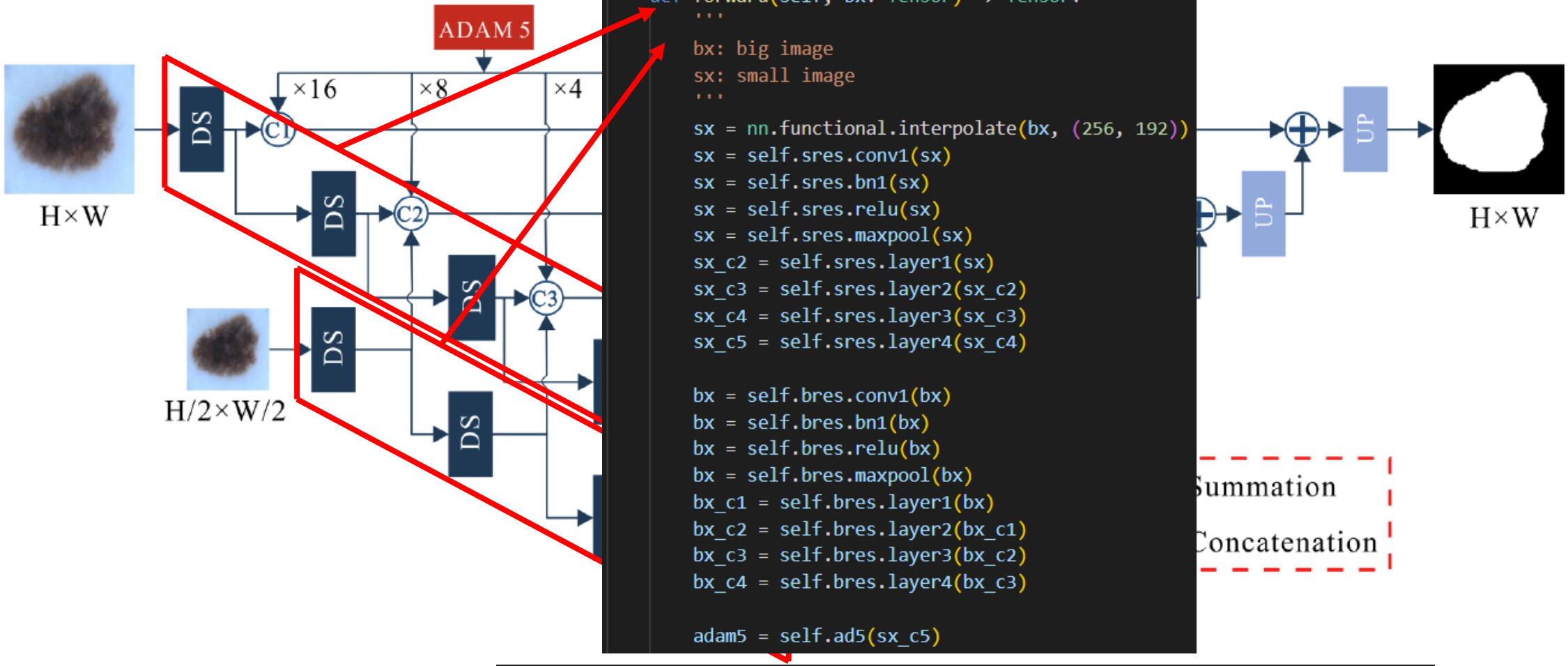

```
sx = nn.functional.interpolate(bx, (192, 256))
```

ARCHITECTURE: BACKBONE

2 ResNet-34 Architecture



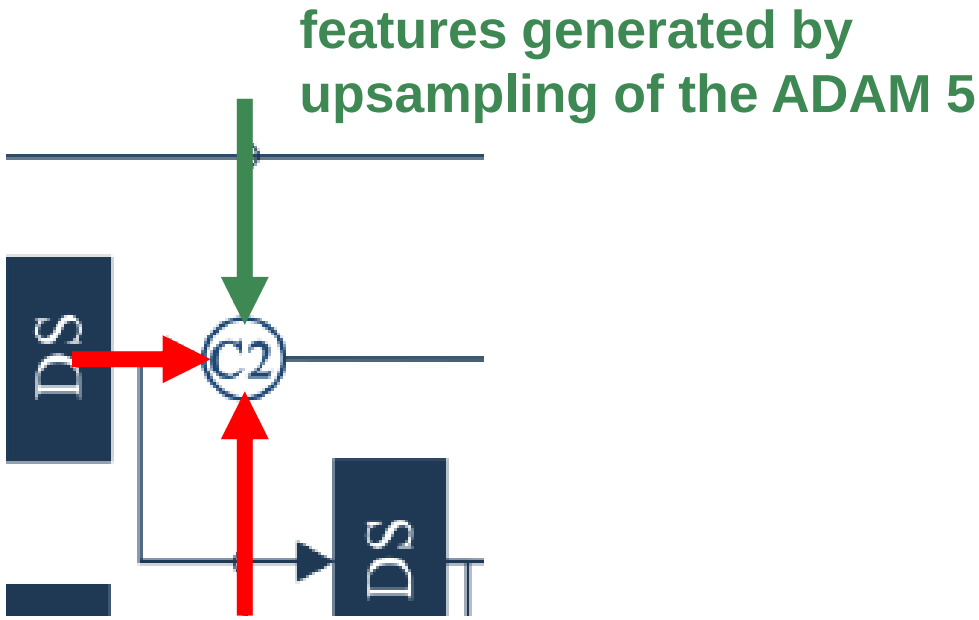
ARCHITECTURE: BA




```python
class ADAMNet_Seg(nn.Module):
    # Using BasicBlock
    def __init__(…

    def forward(self, bx: Tensor) -> Tensor:
        '''
        bx: big image
        sx: small image
        '''
        sx = nn.functional.interpolate(bx, (256, 192))
        sx = self.sres.conv1(sx)
        sx = self.sres.bn1(sx)
        sx = self.sres.relu(sx)
        sx = self.sres.maxpool(sx)
        sx_c2 = self.sres.layer1(sx)
        sx_c3 = self.sres.layer2(sx_c2)
        sx_c4 = self.sres.layer3(sx_c3)
        sx_c5 = self.sres.layer4(sx_c4)

        bx = self.bres.conv1(bx)
        bx = self.bres.bn1(bx)
        bx = self.bres.relu(bx)
        bx = self.bres.maxpool(bx)
        bx_c1 = self.bres.layer1(bx)
        bx_c2 = self.bres.layer2(bx_c1)
        bx_c3 = self.bres.layer3(bx_c2)
        bx_c4 = self.bres.layer4(bx_c3)

        adam5 = self.ad5(sx_c5)
```

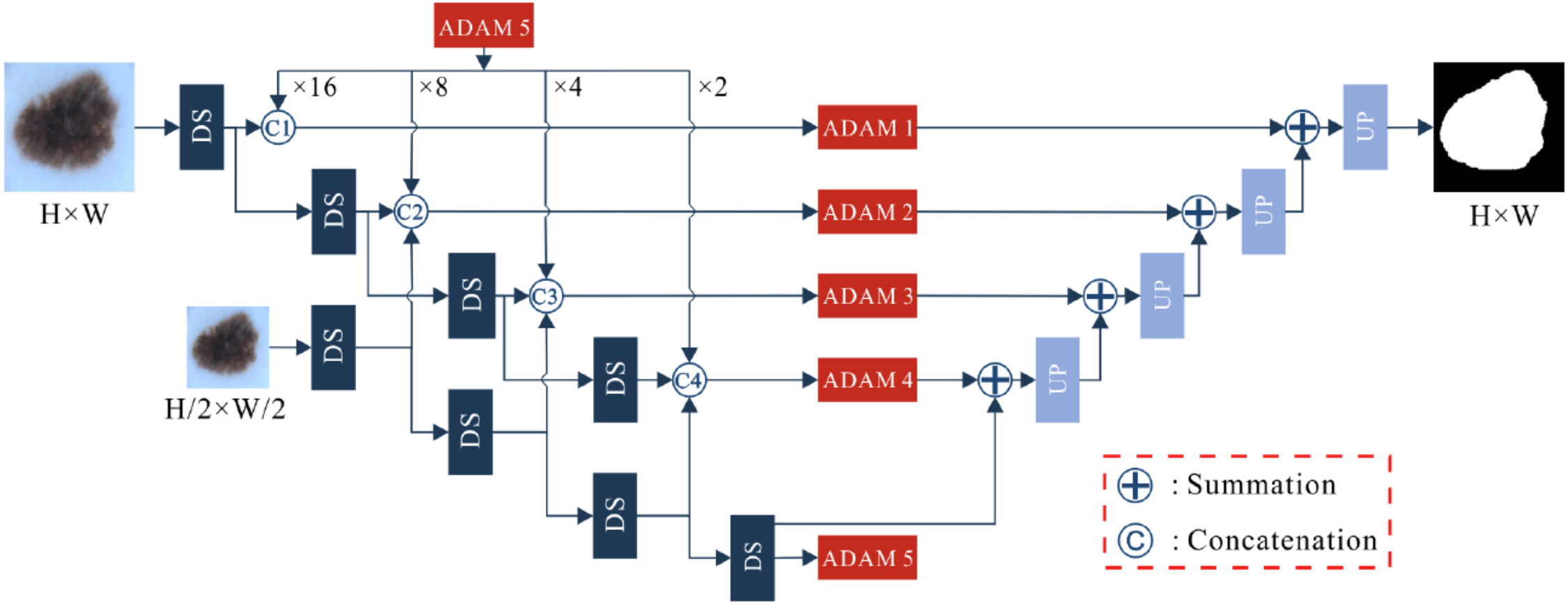
```python
# Using ResNet34 as the backbone network
model = ADAMNet_Seg(8, BasicBlock, [3, 4, 6, 3])
```

ARCHITECTURE: BACKBONE



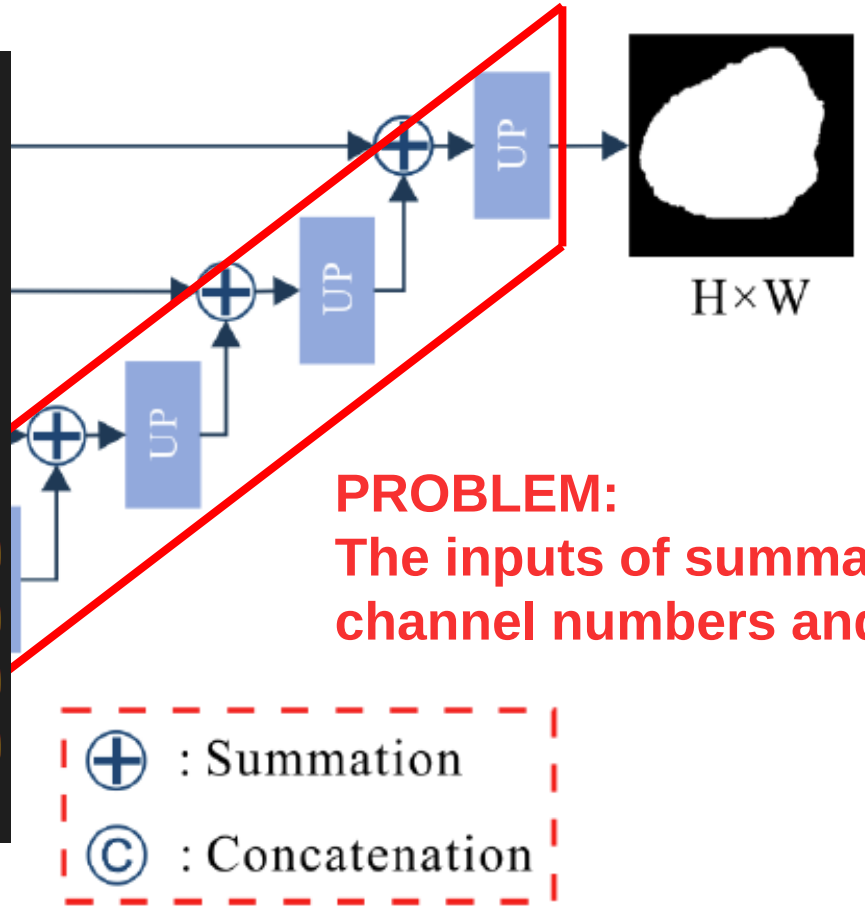

```
c1 = torch.cat([bx_c1, self.Up_to_c1(adam5)], 1)
c2 = torch.cat([bx_c2, sx_c2, self.Up_to_c2(adam5)], 1)
c3 = torch.cat([bx_c3, sx_c3, self.Up_to_c3(adam5)], 1)
c4 = torch.cat([bx_c4, sx_c4, self.Up_to_c4(adam5)], 1)
```

ARCHITECTURE: BACKBONE



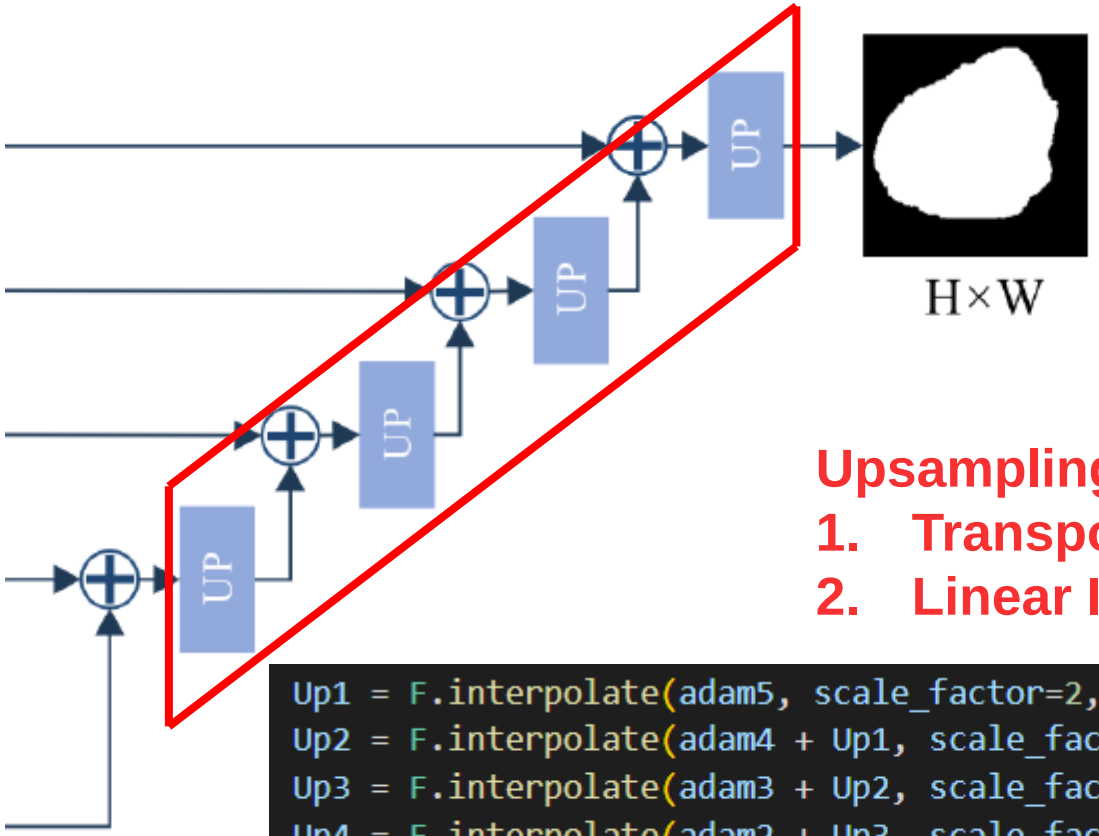
ARCHITECTURE: BACKBONE

```
adam1 = self.conv1(adam1)
adam2 = self.conv2(adam2)
adam3 = self.conv3(adam3)
adam4 = self.conv4(adam4)
adam5 = self.conv5(adam5)

Up1 = self.Upsample(adam5)
Up2 = self.Upsample(adam4 + Up1)
Up3 = self.Upsample(adam3 + Up2)
Up4 = self.Upsample(adam2 + Up3)
Up5 = self.Upsample(adam1 + Up4)
Up6 = self.Upsample(Up5)
```



PROBLEM:
The inputs of summation have different channel numbers and map sizes.

ARCHITECTURE: BACKBONE




```
Up1 = self.Trans(adam5)
Up2 = self.Trans(adam4 + Up1)
Up3 = self.Trans(adam3 + Up2)
Up4 = self.Trans(adam2 + Up3)
Up5 = self.Trans(adam1 + Up4)
Up6 = self.Trans(Up5)

out = self.sigmoid(Up6)
return out
```

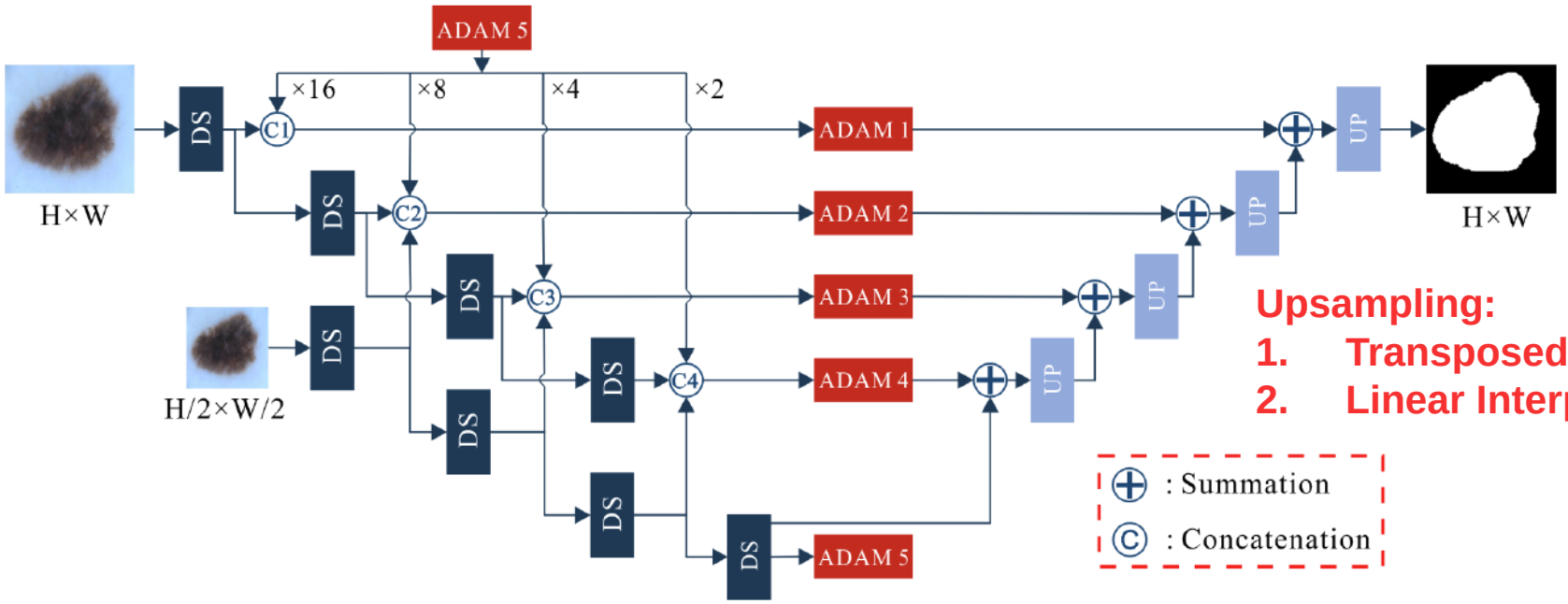
Upsampling:

1. **Transposed Convolution**
2. **Linear Interpolation**

```
Up1 = F.interpolate(adam5, scale_factor=2, mode='bilinear')
Up2 = F.interpolate(adam4 + Up1, scale_factor=2, mode='bilinear')
Up3 = F.interpolate(adam3 + Up2, scale_factor=2, mode='bilinear')
Up4 = F.interpolate(adam2 + Up3, scale_factor=2, mode='bilinear')
Up5 = F.interpolate(adam1 + Up4, scale_factor=2, mode='bilinear')
Up6 = F.interpolate(Up5, scale_factor=2, mode='bilinear')

out = self.sigmoid(Up6)
return out
```

ARCHITECTURE: Comparison Experiment

1. Proposed Architecture with ADAM:

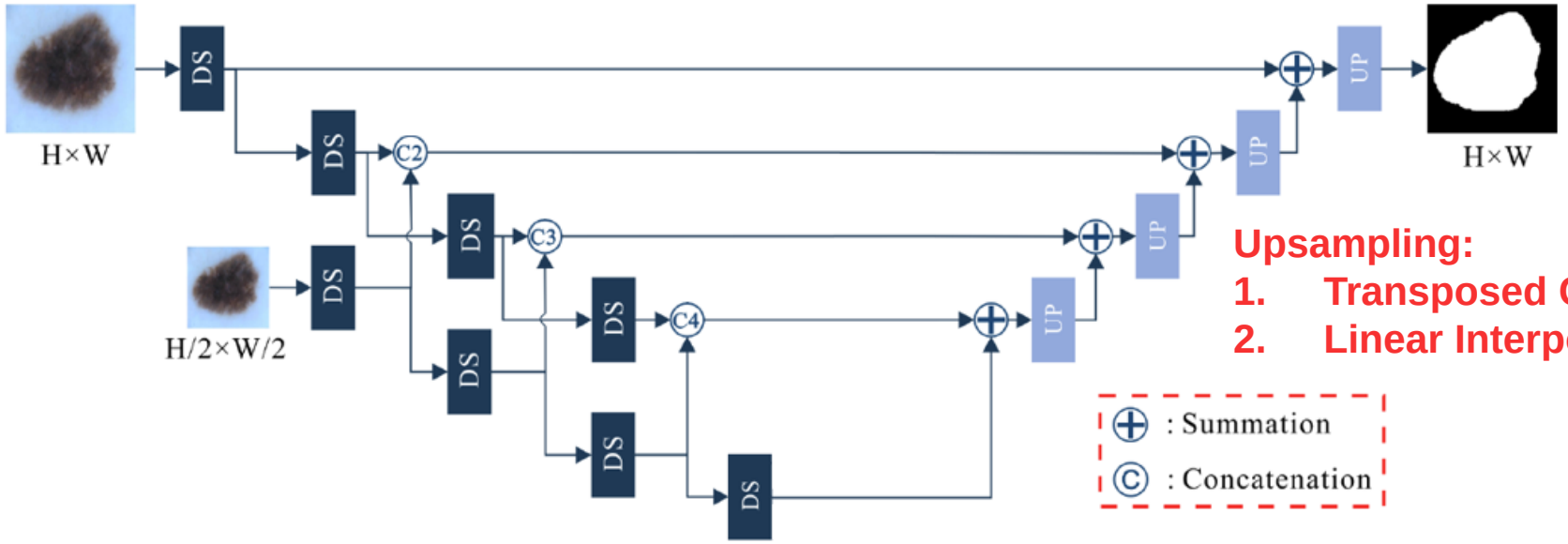


Upsampling:

1. **Transposed Convolution**
2. **Linear Interpolation**

ARCHITECTURE: Comparison Experiment

2. (Comparison Exp) Basic Architecture without ADAM:

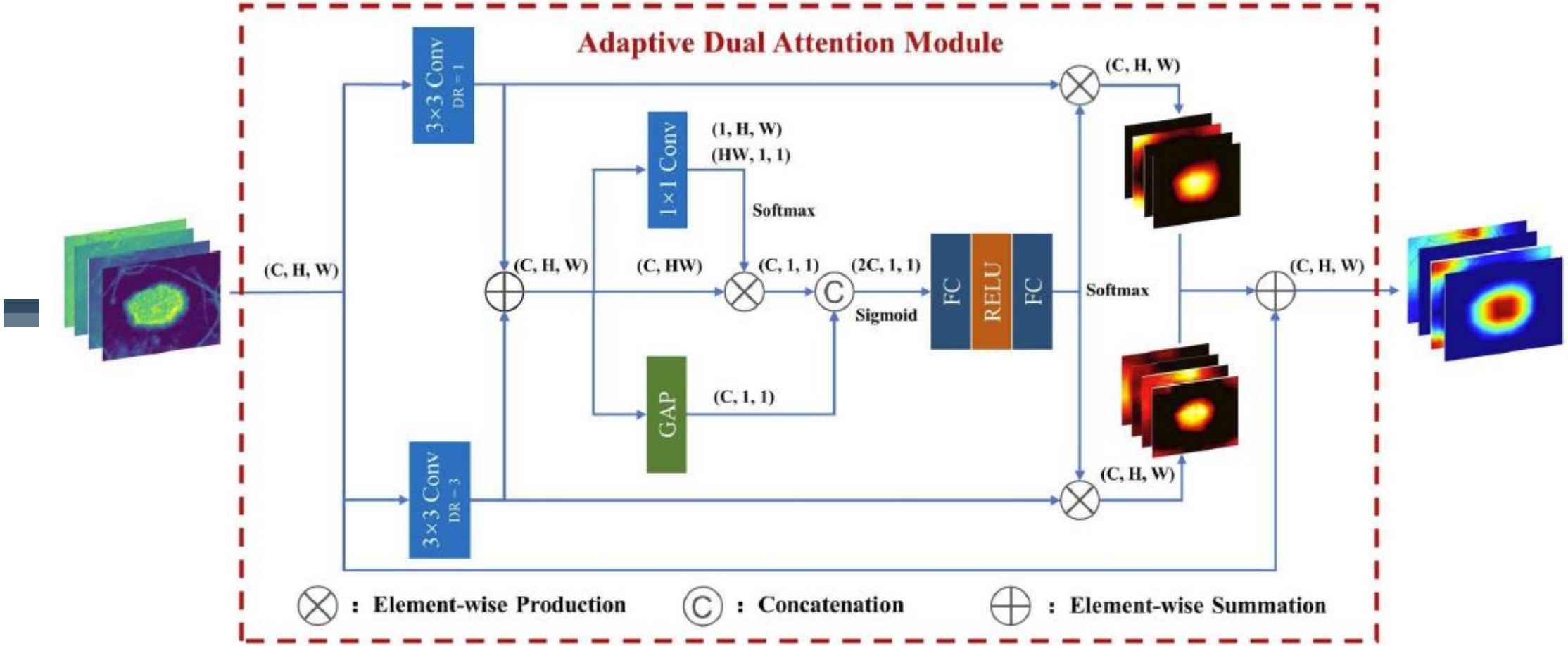


Upsampling:

1. **Transposed Convolution**
2. **Linear Interpolation**

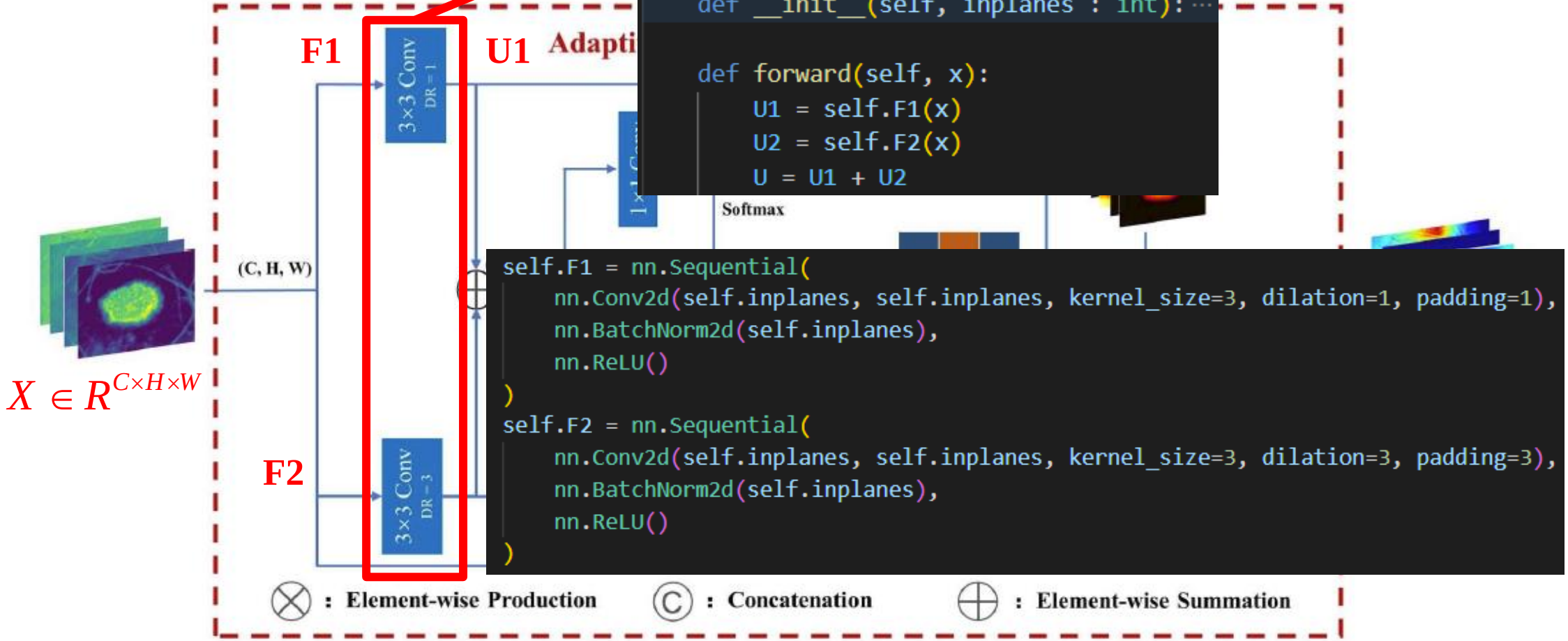
Looks like U-Net

ARCHITECTURE: ADAM



ARCHITECTURE: ADAM

Kernel 3 by 3
Dilation rate: 1 & 3

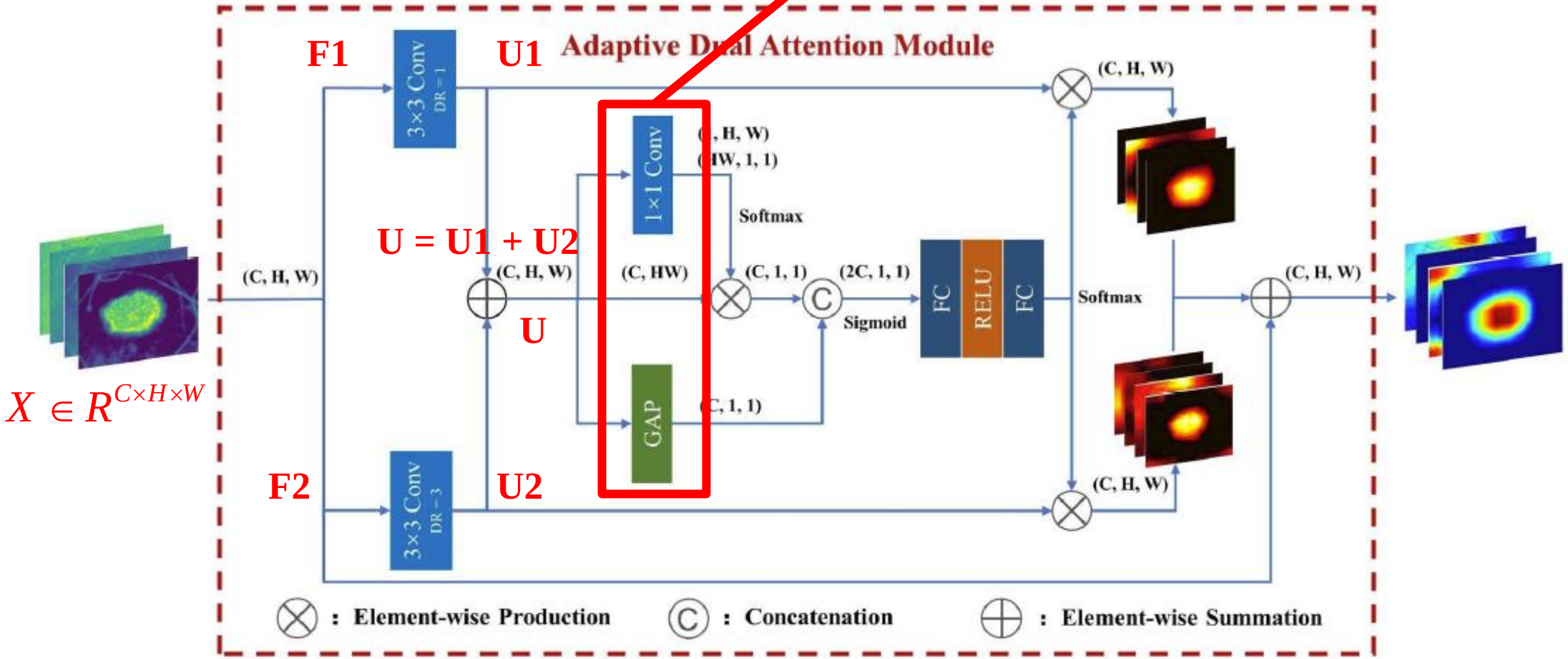



```
class Adam(nn.Module):
    def __init__(self, inplanes : int):···

    def forward(self, x):
        U1 = self.F1(x)
        U2 = self.F2(x)
        U = U1 + U2
```

```
self.F1 = nn.Sequential(
    nn.Conv2d(self.inplanes, self.inplanes, kernel_size=3, dilation=1, padding=1),
    nn.BatchNorm2d(self.inplanes),
    nn.ReLU()
)
self.F2 = nn.Sequential(
    nn.Conv2d(self.inplanes, self.inplanes, kernel_size=3, dilation=3, padding=3),
    nn.BatchNorm2d(self.inplanes),
    nn.ReLU()
)
```

ARCHITECTURE: ADAM

2 different attention mechanisms:
global average pooling (GAP)
pixel-wise correlations (PC)



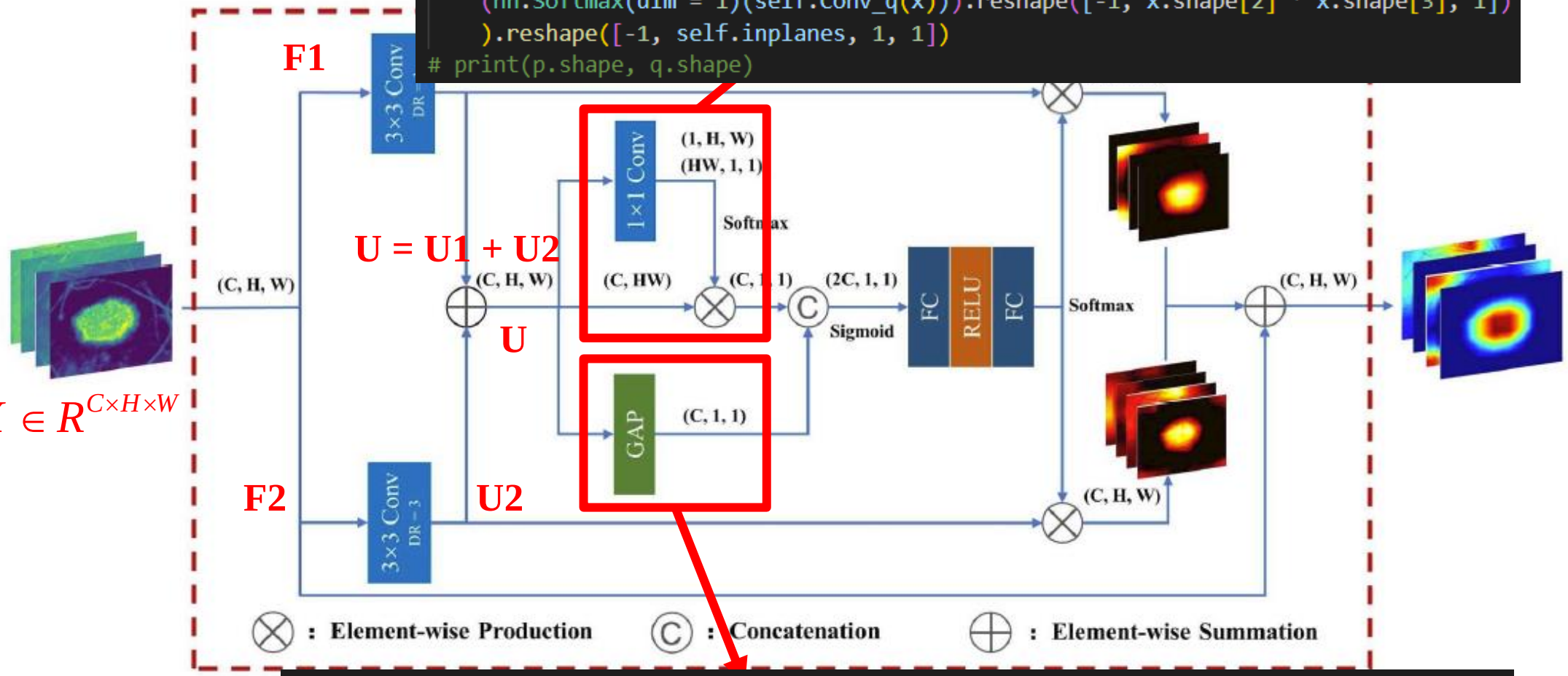
$X \in R^{C \times H \times W}$

ARCHITECTURE: ADAM

$q \in R^{C \times H \times W}$

the cth element of q is calculated by

```
q = torch.matmul(
    U.reshape([-1, self.inplanes, x.shape[2] * x.shape[3]]),
    (nn.Softmax(dim = 1)(self.Conv_q(x))).reshape([-1, x.shape[2] * x.shape[3], 1])
    ).reshape([-1, self.inplanes, 1, 1])
# print(p.shape, q.shape)
```



$X \in R^{C \times H \times W}$

```
p = torch.mean(torch.mean(U, dim=2), dim=2).reshape(-1, self.inplanes, 1, 1)
```

$$p_c = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} U_c(i, j)$$

ARCHITECTURE: ADAM

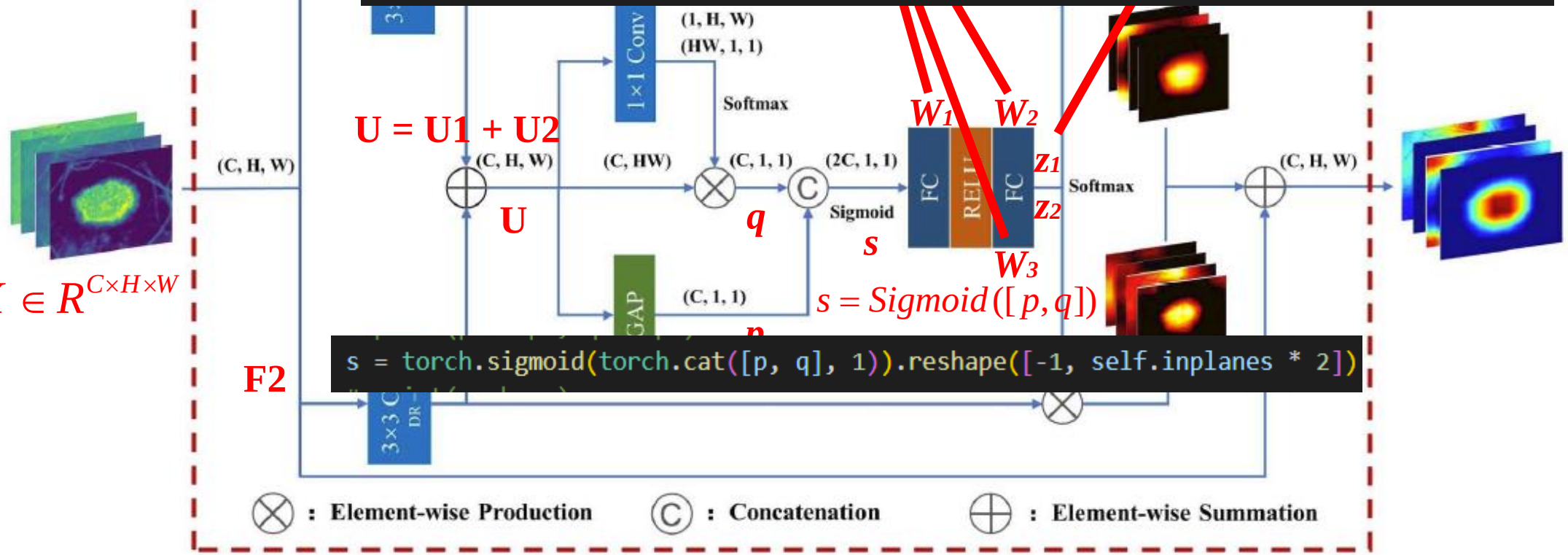
$W_1 \in R^{d \times 2C}$

$W_2, W_3 \in R^{C \times d}$

$d = \max(C/8, 48)$

$z_1, z_2 \in R^{C \times 1 \times 1}$

```
z1 = self.W2(nn.ReLU()(self.W1(s)))
z2 = self.W3(nn.ReLU()(self.W1(s)))
a1 = (torch.exp(z1) / torch.exp(z1) + torch.exp(z2)).reshape([-1, self.inplanes, 1, 1])
a2 = (torch.exp(z2) / torch.exp(z1) + torch.exp(z2)).reshape([-1, self.inplanes, 1, 1])
```



$X \in R^{C \times H \times W}$

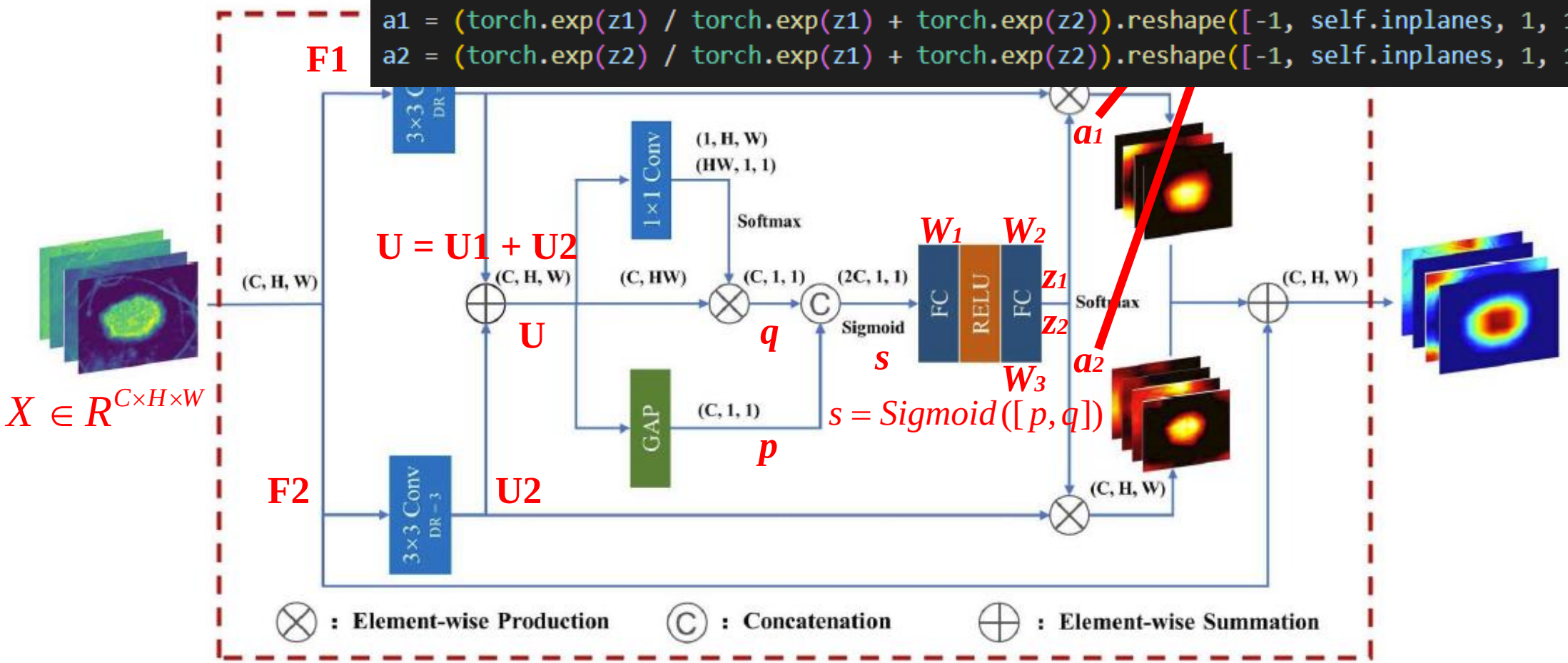
$s = Sigmoid([p, q])$

```
s = torch.sigmoid(torch.cat([p, q], 1)).reshape([-1, self.inplanes * 2])
```

ARCHITECTURE: ADAM

$$a_{1c} = \frac{e^{z_{1c}}}{e^{z_{1c}} + e^{z_{2c}}}, a_{2c} = \frac{e^{z_{2c}}}{e^{z_{1c}} + e^{z_{2c}}}$$

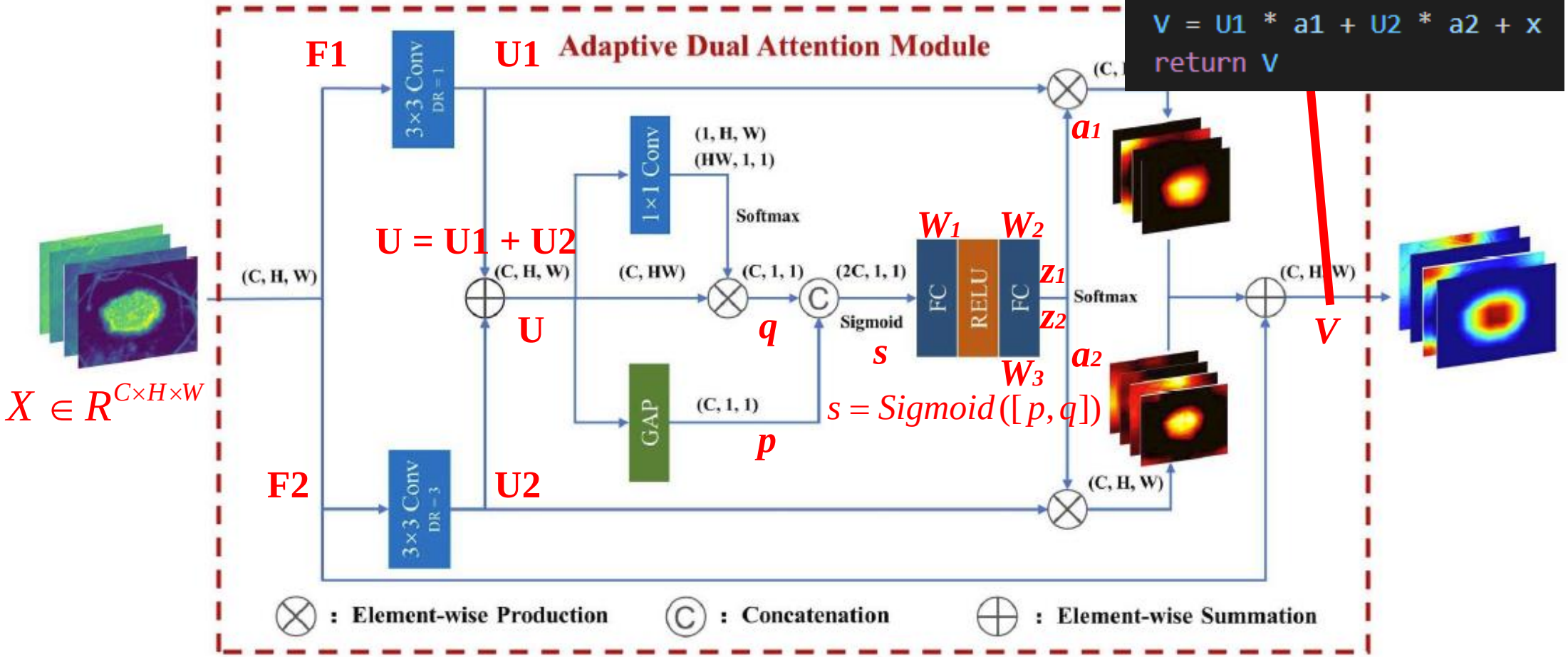
```
z1 = self.W2(nn.ReLU()(self.W1(s)))
z2 = self.W3(nn.ReLU()(self.W1(s)))
a1 = (torch.exp(z1) / torch.exp(z1) + torch.exp(z2)).reshape([-1, self.inplanes, 1, 1])
a2 = (torch.exp(z2) / torch.exp(z1) + torch.exp(z2)).reshape([-1, self.inplanes, 1, 1])
```



ARCHITECTURE: ADAM

$$V_c = a_{1c} \cdot U_{1c} + a_{2c} \cdot U_{2c} + X_c$$

```
V = U1 * a1 + U2 * a2 + x
return V
```



OUTLINE

This presentation is divided into 6 sections:

1. The Architecture
2. Dataset and Dataloader ☜
3. Data Augmentation
4. Training
5. Evaluating Indicator
6. Experiment Results & Analysis

DATASET & DATALOADER

```python
class ISICDataset_Seg(data.Dataset):
    def __init__(self, data_path, mask_path):  # 载入数据…

    def __getitem__(self, item):  # 返回相应位置的数据 & 标签
        # return self._x[item], self._y[item]
        image_pa = self.images[item]
        mask_pa = self.masks[item]

        img = cv.imread(image_pa)
        img1 = cv.cvtColor(img, cv.COLOR_BGR2RGB)
        # img2 = cv.cvtColor(img, cv.COLOR_BGR2HSV)[:,:,1:3]  # drop the H channel
        img2 = cv.cvtColor(img, cv.COLOR_BGR2HSV)
        img3 = cv.cvtColor(img, cv.COLOR_BGR2LAB)
        img1 = Image.fromarray(img1)
        img2 = Image.fromarray(img2)
        img3 = Image.fromarray(img3)
        img1 = self.img_transform(img1)
        img2 = self.img_transform(img2)
        img3 = self.img_transform(img3)
        img1 = np.array(img1)
        img2 = np.array(img2)[1:3,:,:]  # drop the H channel
        img3 = np.array(img3)
        image = np.concatenate([img1, img2, img3], axis=0)
        image = torch.from_numpy(image)

        ma = cv.imread(mask_pa)
        ma = Image.fromarray(ma)
        ma = self.mask_transform(ma)
        ma = np.array(ma)
        mask = torch.from_numpy(ma)[0:1,:,:]
        return image, mask

    def __len__(self):  # 返回整个数据集的长度
        return self._len
```

For Segmentation

```python
label_dict = {'[0. 0.]': 2, '[1. 0.]': 0, '[0. 1.]': 1}
class ISICDataset_Cla(data.Dataset):
    def __init__(self, data_path, label_path):  # 载入数据…

    def __getitem__(self, item):  # 返回相应位置的数据 & 标签
        # return self._x[item], self._y[item]
        image_pa = self.images[item]
        img = cv.imread(image_pa)

        seed = np.random.randint(2147483647)
        random.seed(seed)
        img = Image.fromarray(img)
        img = self.augmentation(img)
        img = np.array(img)
        img = Image.fromarray(img)
        img = self.img_transform(img)
        img = np.array(img)
        image = torch.from_numpy(img)
        label = torch.LongTensor(np.array([self.csv[item]]))[0]
        return image, label

    def __len__(self):  # 返回整个数据集的长度
        return self._len
```

For Classification

DATASET & DATALOADER

```python
# To make sure a pair of image & mask are randomly augmented in the same way,
# here I pack these transform together.
def augment(img, mask):
    # Transform lib is often used to process PIL image.
    img = Image.fromarray(img)
    mask = Image.fromarray(mask)

    img = transforms.Resize((384, 512), interpolation=InterpolationMode.BICUBIC)(img)
    mask = transforms.Resize((384, 512), interpolation=InterpolationMode.BICUBIC)(mask)

    H, W = mask.size[0], mask.size[1]

    if random.random() > 0.5:  # Horizontal flipping
        img = tf.hflip(img)
        mask = tf.hflip(mask)

    if random.random() > 0.5:  # Vertical flipping
        img = tf.vflip(img)
        mask = tf.vflip(mask)

    scale_ratio = random.uniform(0.6,1.3)  # Scaling (range from 0.6 to 1.3) and Cropping
    img = transforms.Resize((int(scale_ratio*H), int(scale_ratio*W)))(img)
    mask = transforms.Resize((int(scale_ratio*H), int(scale_ratio*W)))(mask)
    img = transforms.CenterCrop((H, W))(img)
    mask = transforms.CenterCrop((H, W))(mask)
```

```python
angle = transforms.RandomRotation.get_params([0, 180])
img = img.rotate(angle)
mask = mask.rotate(angle)

img = np.array(img)
mask = np.array(mask)

return img, mask
```

DATASET & DATALOADER

For Segmentation

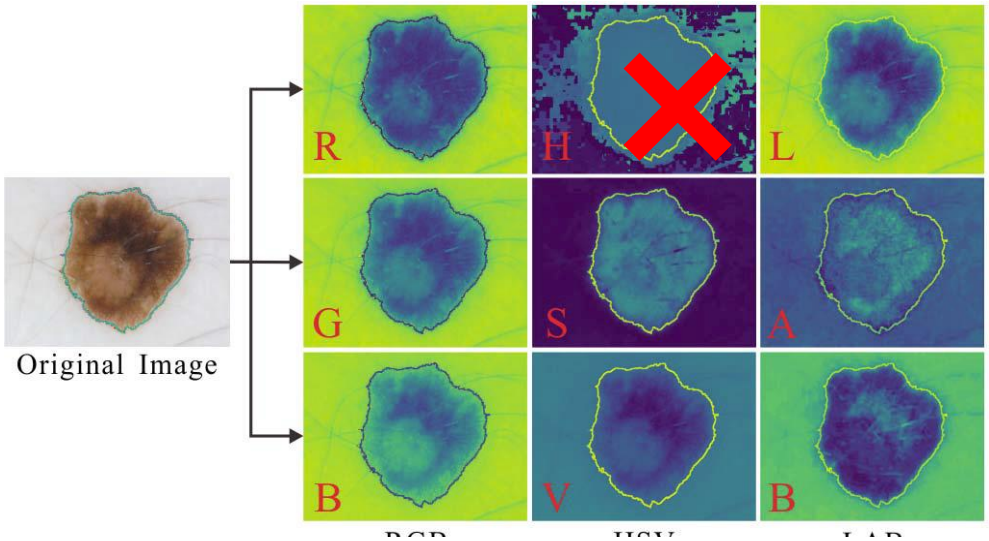


TABLE IV

ACCURACY COMPARISONS WITH DIFFERENT INPUT CHANNELS

# of Input Channel	TJI	JSI	DSC	SE	SP	ACC
3-Channel	80.92	84.11	90.73	**92.72**	96.30	96.12
8-Channel	**81.73**	**84.70**	**91.63**	92.56	**97.42**	**96.36**

```python
def __getitem__(self, item):  # 返回相应位置的数据 & 标签
    # return self._x[item], self._y[item]
    image_pa = self.images[item]
    mask_pa = self.masks[item]

    img = cv.imread(image_pa)
    img1 = cv.cvtColor(img, cv.COLOR_BGR2RGB)
    # img2 = cv.cvtColor(img, cv.COLOR_BGR2HSV)[:,:,1:3]  # drop the H channel
    img2 = cv.cvtColor(img, cv.COLOR_BGR2HSV)
    img3 = cv.cvtColor(img, cv.COLOR_BGR2LAB)
    img1 = Image.fromarray(img1)
    img2 = Image.fromarray(img2)
    img3 = Image.fromarray(img3)
    img1 = self.img_transform(img1)
    img2 = self.img_transform(img2)
    img3 = self.img_transform(img3)
    img1 = np.array(img1)
    2 = np.array(img2)[1:3,:,:]  # drop the H channel
    3 = np.array(img3)
    ge = np.concatenate([img1, img2, img3], axis=0)
    ge = torch.from_numpy(image)

    = cv.imread(mask_pa)
    = Image.fromarray(ma)
    = self.mask_transform(ma)
    = np.array(ma)
    k = torch.from_numpy(ma)[0:1,:,:]
    return image, mask
```

OUTLINE

This presentation is divided into 6 sections:

1. The Architecture
2. Dataset and Dataloader
3. Data Augmentation ☜
4. Training
5. Evaluating Indicator
6. Experiment Results & Analysis

DATA AUGMENTATION

1. Vertical & Horizontal flipping
2. Rotations (randomly from 0 to 180 degrees)
3. Scaling (range from 0.6 to 1.3)
4. Random cropping
5. Sharpening
6. Adjustments of color distributions (brightness, contrast, saturation, chroma of images)
7. Adding noises (Gaussian, Salt or pepper noises)

Generate more than ten times of samples than the original ISIC2017.

OUTLINE

This presentation is divided into 6 sections:

1. The Architecture
2. Dataset and Dataloader
3. Data Augmentation
4. Training ☜
5. Evaluating Indicator
6. Experiment Results & Analysis

TRAINING: Parameters of dataloader, optimizer & loss

Initialization: Kaiming Initialization
Optimizer: Adam
Learning rate: 0.0002
Epoches: 250
Batch size: 16
Loss function: L_{seg} (the combination of dice loss & bce loss)

$$L_{dice} = 1 - \boxed{2\frac{\sum_i y_i p_i}{\sum_i y_i + \sum_i p_i}}$$ **Dice Coefficient**

$$L_{bce} = -\sum_i \left(y_i \ln(p_i) + (1 - y_i)\ln(1 - p_i)\right)$$

$$L_{seg} = L_{bce} + L_{dice}$$

TRAINING: Parameters of dataloader, optimizer & loss

```python
class BinaryDiceLoss(nn.Module):
    def __init__(self):
        super(BinaryDiceLoss, self).__init__()

    def forward(self, pred, label):
        batch_size = label.size()[0]  # fetch batch_size
        smooth = 100  # smooth variable (in case of denominator = 0)

        pred_flat = pred.view(batch_size, -1)
        label_flat = label.view(batch_size, -1)

        intersection = pred_flat * label_flat
        N_dice_eff = (2 * intersection.sum(1) + smooth) / (pred_flat.sum(1) + label_flat.sum(1) + smooth)

        loss = 1 - N_dice_eff.sum() / batch_size  # the average loss of this batch
        return loss
```

```python
# L_DiceCE = L_CE + L_Dice
class DiceCELoss(nn.Module):
    def __init__(self):
        super(DiceCELoss, self).__init__()
        self.dice = BinaryDiceLoss()
        # self.bce = BinaryCrossEntropyLoss()
        self.bce = nn.BCELoss()

    def forward(self, pred, label):
        loss = self.dice(pred, label) + self.bce(pred, label)
        return loss
```

TRAINING: Load pre-trained ResNet-34

```
# load pretrained ResNet34 model
net_dict = model.state_dict()
snet_dict = model.sres.state_dict()
bnet_dict = model.bres.state_dict()
predict_model = torch.load('resnet34-333f7ec4.pth')
# find shared layer parameters
sstate_dict = {k: v for k, v in predict_model.items() if k in snet_dict.keys()}
# delete non-shared parameters
sstate_dict.pop('fc.weight')
sstate_dict.pop('fc.bias')
sstate_dict.pop('conv1.weight')

# find shared layer parameters
bstate_dict = {k: v for k, v in predict_model.items() if k in bnet_dict.keys()}
# delete non-shared parameters
bstate_dict.pop('fc.weight')
bstate_dict.pop('fc.bias')
bstate_dict.pop('conv1.weight')

# load pre-trained parameters into two ResNet34 in ADAMNet
snet_dict.update(sstate_dict)
bnet_dict.update(bstate_dict)
model.sres.load_state_dict(snet_dict)
model.bres.load_state_dict(bnet_dict)

model = model.to(device)
```

Load twice & Pop out the first 3-kernel Conv layer.

OUTLINE

This presentation is divided into 6 sections:

1. The Architecture
2. Dataset and Dataloader
3. Data Augmentation
4. Training
5. Evaluating Indicator ☜
6. Experiment Results & Analysis

EVALUATING INDICATOR

$$\mathrm{JSI} = \frac{|P \cap G|}{|P \cup G|},$$

$$\mathrm{DSC} = \boxed{\frac{2|P \cap G|}{|P| + |G|},}$$ **Dice Coefficient**

$$\mathrm{SE} = \mathrm{TP}/(\mathrm{TP} + \mathrm{FN}),$$ **Recall**

$$\mathrm{SP} = \mathrm{TN}/(\mathrm{TN} + \mathrm{FP}),$$

$$\mathrm{ACC} = (\mathrm{TP} + \mathrm{TN})/(\mathrm{TP} + \mathrm{FP} + \mathrm{TN} + \mathrm{FN}),$$

$$\mathrm{TJI} = \begin{cases} \mathrm{JSI} & \mathrm{JSI} \geq 0.65 \\ 0 & \mathrm{JSI} < 0.65, \end{cases}$$

where P is the prediction and G is the ground truth. TP (TN) is the number of pixels that are correctly predicted and labelled as positive (negative).

EVALUATING INDICATOR

TABLE I

STATISTICAL COMPARISONS OF ABLATION STUDIES

Model	TJI	JSI	DSC	SE	SP	ACC
U-Net	71.56	79.28	87.07	87.99	97.31	94.87
DEA	79.45	82.67	90.21	90.56	97.38	95.76
DEA + GAP	80.51	82.83	90.38	91.97	97.03	95.81
DEA + PC	80.76	83.16	90.63	92.08	97.11	95.83
ADAM w/o AAW	80.87	84.28	90.77	92.30	96.61	96.24
Ours	**81.73**	**84.70**	**91.63**	**92.56**	**97.42**	**96.36**

TABLE VIII

ACCURACY COMPARISONS WITH DIFFERENT SKIN LESION SEGMENTATION METHODS IN THE COMPETITION LEADER BOARDS

Method	Year	TJI	JSI	DSC	SE	SP	ACC
ISBI2017 Testset							
Bi et al. [47]	2017	-	76.0	84.4	80.2	**98.5**	93.4
Li et al. [46]	2017	-	76.2	84.7	82.0	97.8	93.2
Yuan et al. [45]	2017	-	76.5	84.9	82.5	97.5	93.4
FrCN [48]	2018	-	77.1	87.0	85.4	96.7	94.3
LCASA-Net [49]	2019	-	80.9	87.9	-	-	94.7
Ours	2019	**78.5**	**82.5**	**89.6**	**90.6**	96.2	**95.7**

TABLE V

ACCURACY AND COMPUTATIONAL COMPLEXITY COMPARISONS WITH DIFFERENT STATE-OF-THE-ART METHODS

Method	Year	ISBI2017 Test Dataset						ISIC2018 Test Dataset						Params(M)	GFLOPs
		TJI	JSI	DSC	SE	SP	ACC	TJI	JSI	DSC	SE	SP	ACC		
U-Net	2015	62.49	73.59	82.42	82.27	**96.94**	92.76	72.8	79.0	86.9	89.7	95.7	92.8	**31**	223
R2U-Net	2018	71.73	78.60	86.50	85.75	96.17	94.34	77.7	83.1	89.5	91.8	92.0	93.1	75	78
SLSDeep	2018	72.45	78.67	86.73	88.98	95.67	94.76	78.1	83.2	89.4	90.4	93.3	93.7	41	**27**
DeepLab V3+	2018	73.22	78.95	86.64	88.64	95.04	94.24	76.3	81.6	88.6	89.3	**97.1**	93.5	59	67
Attention U-Net	2018	74.35	79.79	87.32	88.13	96.03	94.77	77.6	83.0	89.4	92.9	91.8	93.3	51	55
CBAM-Net	2018	74.50	80.25	87.80	87.96	96.12	94.79	77.4	82.5	89.3	93.8	92.3	93.3	32	34
BAM-Net	2018	74.52	80.31	87.94	89.28	95.64	94.96	77.5	82.7	89.4	**95.5**	90.6	93.4	33	34
CE-Net	2019	74.61	80.53	88.02	88.67	95.75	94.93	78.3	83.1	89.8	93.5	93.4	94.1	34	28
DA-Net	2019	75.80	80.93	88.38	89.02	96.13	95.41	78.4	82.8	89.6	93.2	93.7	94.0	50	45
Ours	2019	**78.50**	**82.55**	**89.69**	**90.61**	96.28	**95.70**	**80.4**	**84.4**	**90.8**	94.2	94.1	**94.7**	38	33

EVALUATING INDICATOR

```python
def DSC(pred, label):
    batch_size = label.size()[0]  # fetch batch_size
    smooth = 0.01  # smooth variable (in case of denominator = 0)

    pred_flat = pred.view(batch_size, -1)
    label_flat = label.view(batch_size, -1)

    intersection = pred_flat * label_flat
    N_dice = (2 * intersection.sum(1) + smooth) / (pred_flat.sum(1) + label_flat.sum(1) + smooth)

    out = N_dice.sum() / batch_size
    return out

def JSI(pred, label):
    batch_size = label.size()[0]  # fetch batch_size

    pred_flat = pred.view(batch_size, -1)
    label_flat = label.view(batch_size, -1)

    intersection = pred_flat * label_flat
    N_jsi = intersection.sum(1) / (pred_flat.sum(1) + label_flat.sum(1) - intersection.sum(1))

    out = N_jsi.sum() / batch_size
    return out
```

EVALUATING INDICATOR

```python
def TJI(pred, label):
    batch_size = label.size()[0]  # fetch batch_size

    pred_flat = pred.view(batch_size, -1)
    label_flat = label.view(batch_size, -1)

    intersection = pred_flat * label_flat
    N_jsi = intersection.sum(1) / (pred_flat.sum(1) + label_flat.sum(1) - intersection.sum(1))
    mask = N_jsi >= 0.65
    N_jsi = N_jsi * mask
    out = N_jsi.sum() / batch_size
    return out

def SE(pred, label):
    batch_size = label.size()[0]  # fetch batch_size
    pred_flat = pred.view(batch_size, -1)
    label_flat = label.view(batch_size, -1)

    pred_flat = pred_flat >= 0.5
    tp_flat = pred_flat * label_flat

    N_se = tp_flat.sum(1) / label_flat.sum(1)
    out = N_se.sum() / batch_size
    return out
```

EVALUATING INDICATOR

```python
def SP(pred, label):
    batch_size = label.size()[0]  # fetch batch_size
    pred_flat = pred.view(batch_size, -1)
    label_flat = label.view(batch_size, -1)

    neg_pred_flat = pred_flat < 0.5
    neg_label_flat = label_flat < 0.5

    tn_flat = neg_pred_flat * neg_label_flat

    N_sp = tn_flat.sum(1) / neg_label_flat.sum(1)
    out = N_sp.sum() / batch_size
    return out

def ACC(pred, label):
    batch_size = label.size()[0]  # fetch batch_size
    pred_flat = pred.view(batch_size, -1)
    label_flat = label.view(batch_size, -1)

    neg_pred_flat = pred_flat < 0.5
    neg_label_flat = label_flat < 0.5

    tp_flat = pred_flat * label_flat
    tn_flat = neg_pred_flat * neg_label_flat

    N_acc = (tp_flat.sum(1) + tn_flat.sum(1)) / (label_flat.sum(1) + neg_label_flat.sum(1))
    out = N_acc.sum() / batch_size
    return out
```

OUTLINE

This presentation is divided into 6 sections:

1. The Architecture
2. Dataset and Dataloader
3. Data Augmentation
4. Training
5. Evaluating Indicator
6. Experiment Results & Analysis ☜

Experiment Results & Analysis:

Standard Architecture (the best one in all our experiments)**:**

pre-trained model with ADAM & Trans Conv Up-sample

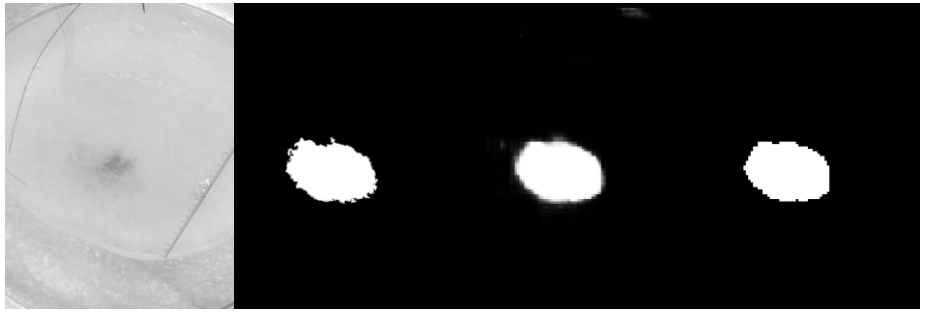
Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.879	0.912	0.944	0.938	0.989	0.983
Test Set	0.811	0.810	0.906	0.917	0.983	0.959

Experiment Results & Analysis:

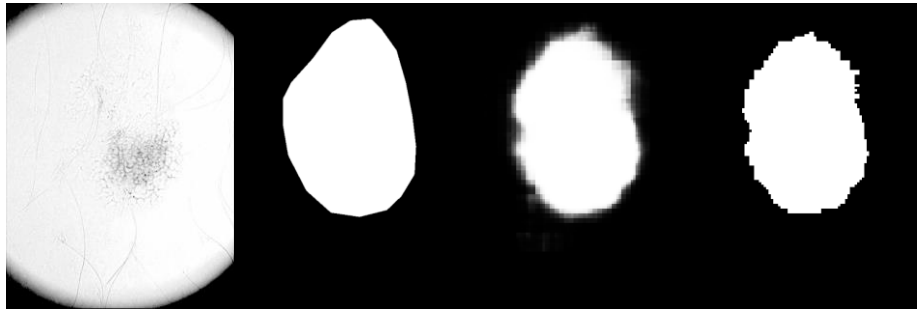
Standard Architecture (the best one in all our experiments)**:**

pre-trained model with ADAM & Trans Conv Up-sample

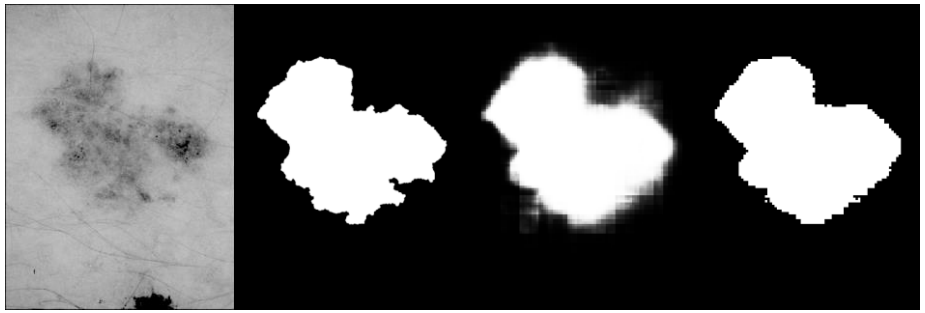
Test set prediction samples (image, label, pred, bpred)



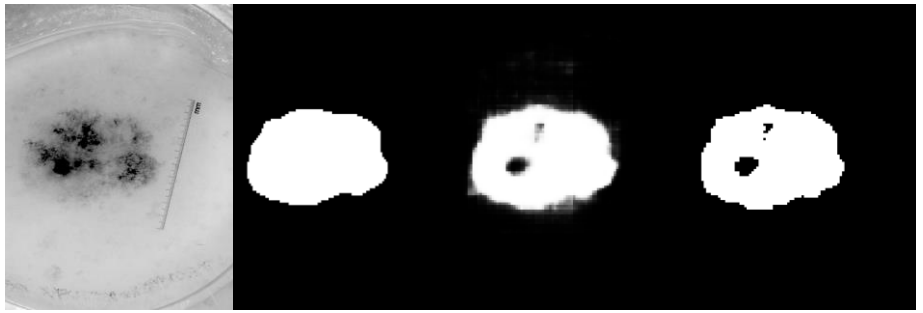
Small samples



Large samples

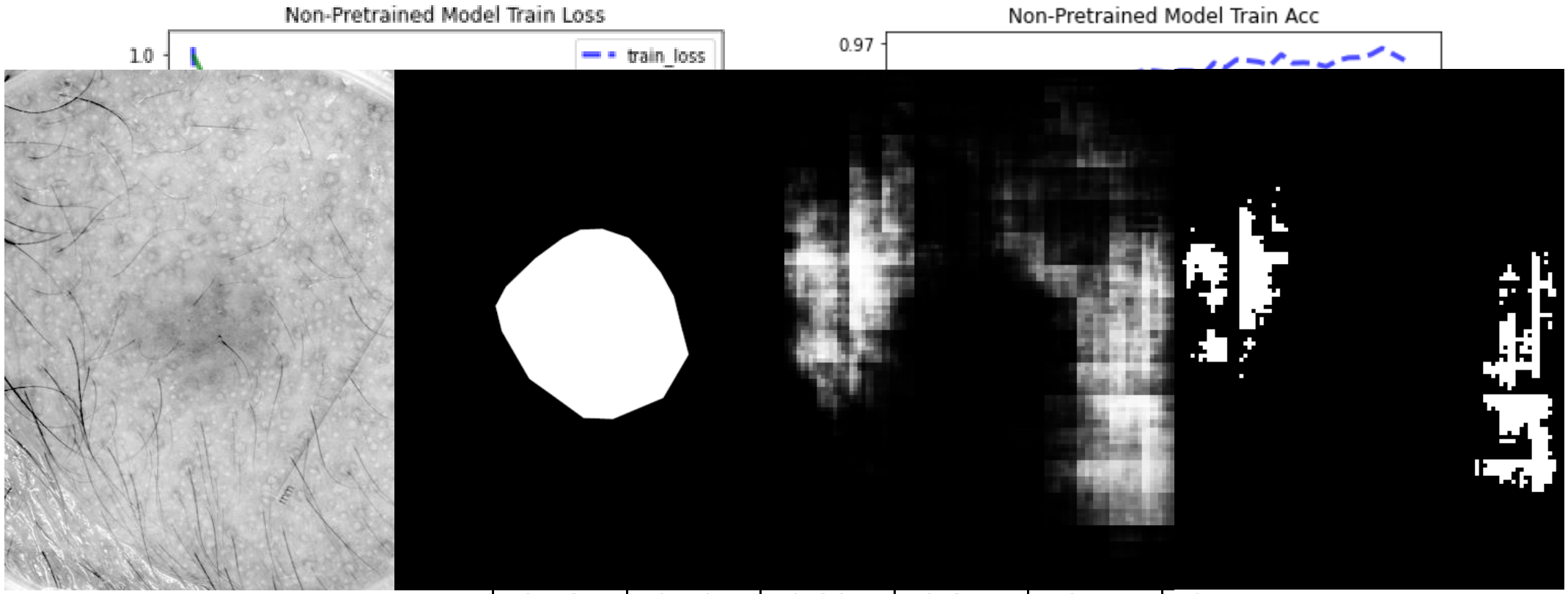


Irregular samples



Imperfect prediction

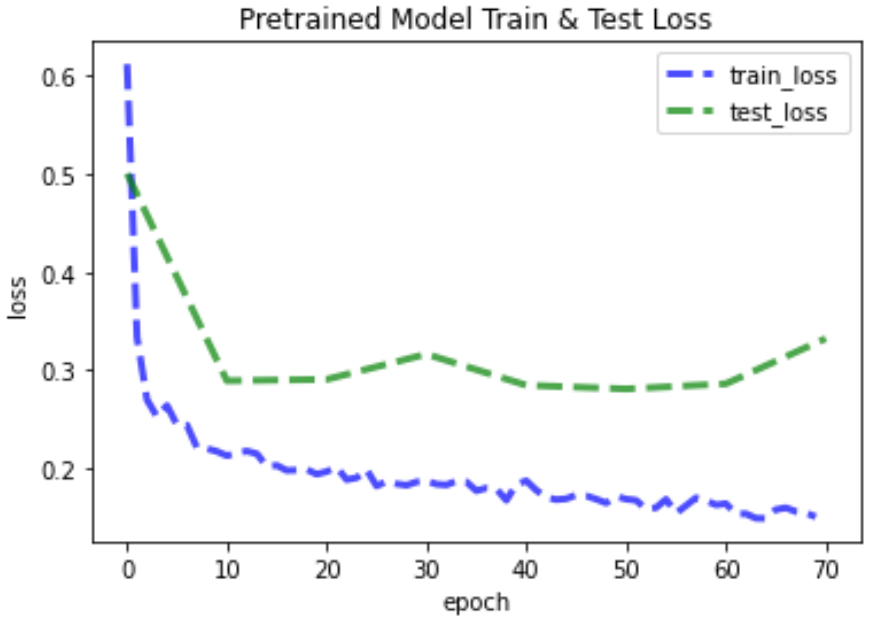
Experiment Results & Analysis:
Comparison Test 1: load pre-trained model & not

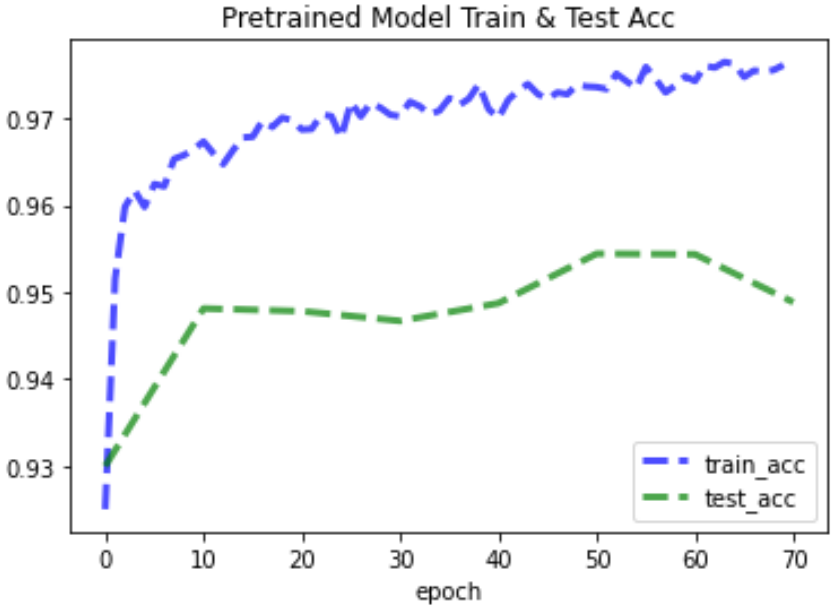


Test Set	0.586	0.709	0.809	0.814	0.97	0.933

Performance of non-pretrained model

Experiment Results & Analysis:
Comparison Test 1: load pre-trained model & not

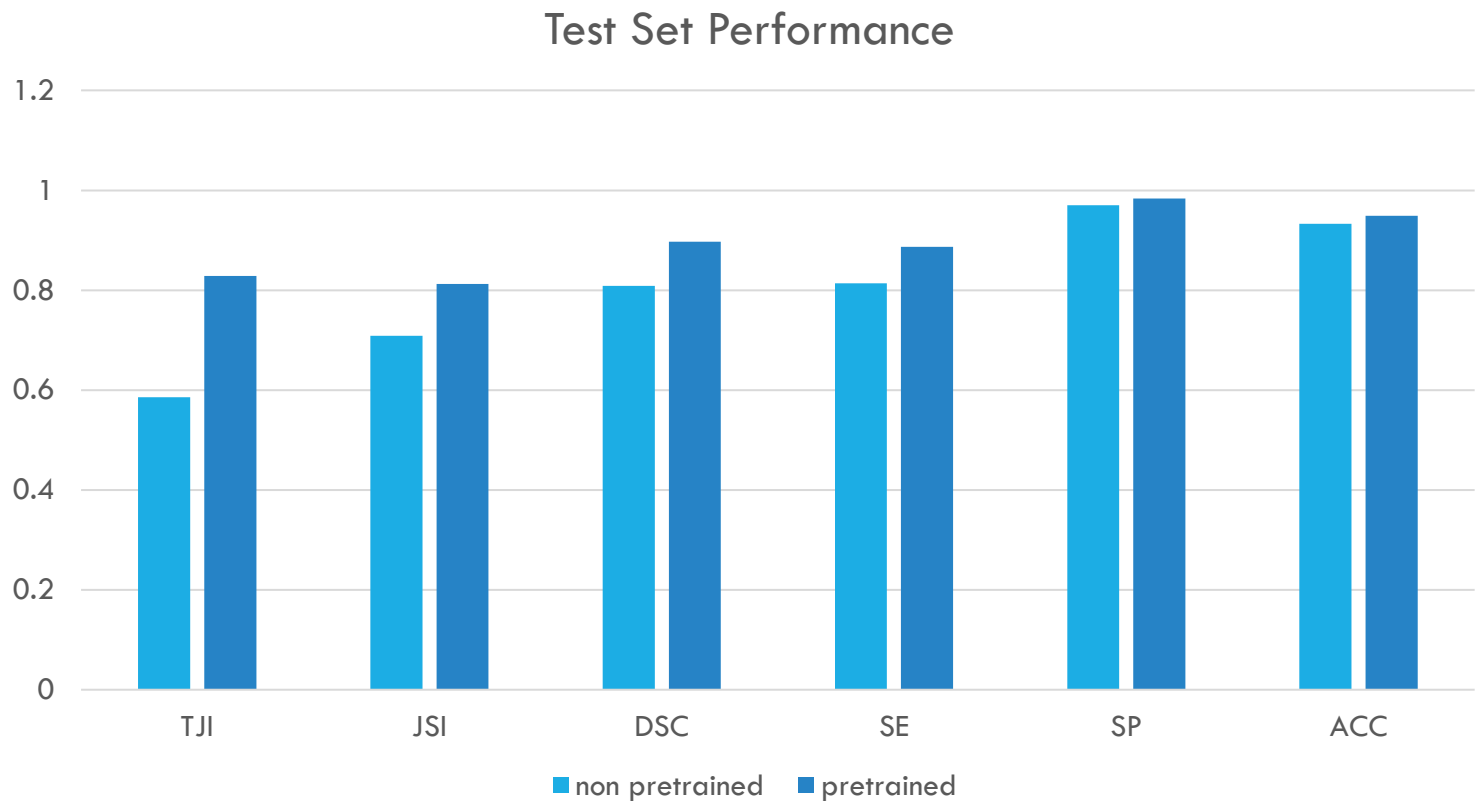




Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.885	0.906	0.937	0.924	0.985	0.976
Test Set	0.829	0.813	0.897	0.887	0.984	0.949

Performance of pretrained model

Experiment Results & Analysis:
Comparison Test 1: load pre-trained model & not



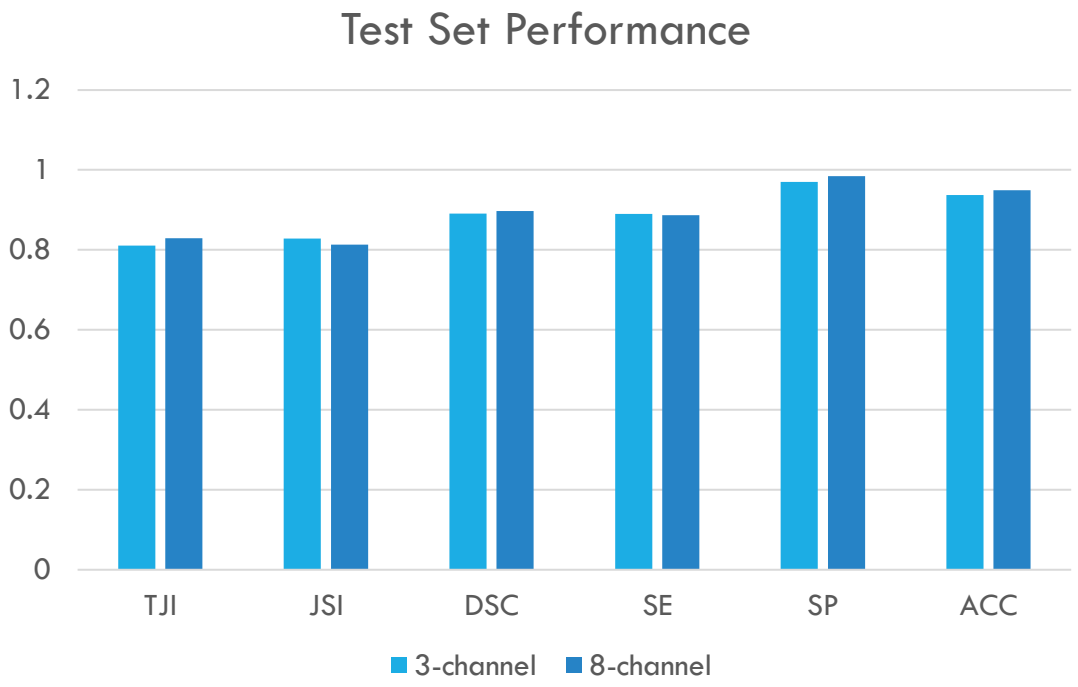
Experiment Results & Analysis:
Comparison Test 2: input 3-channel & 8-channel images

Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.912	0.907	0.940	0.920	0.984	0.971
Test Set	0.811	0.828	0.891	0.890	0.977	0.937

Performance of 3-channel-input model

Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.885	0.906	0.937	0.924	0.985	0.976
Test Set	0.829	0.813	0.897	0.887	0.984	0.949

Performance of 8-channel-input model



Experiment Results & Analysis:
Comparison Test 3: add ADAM & not

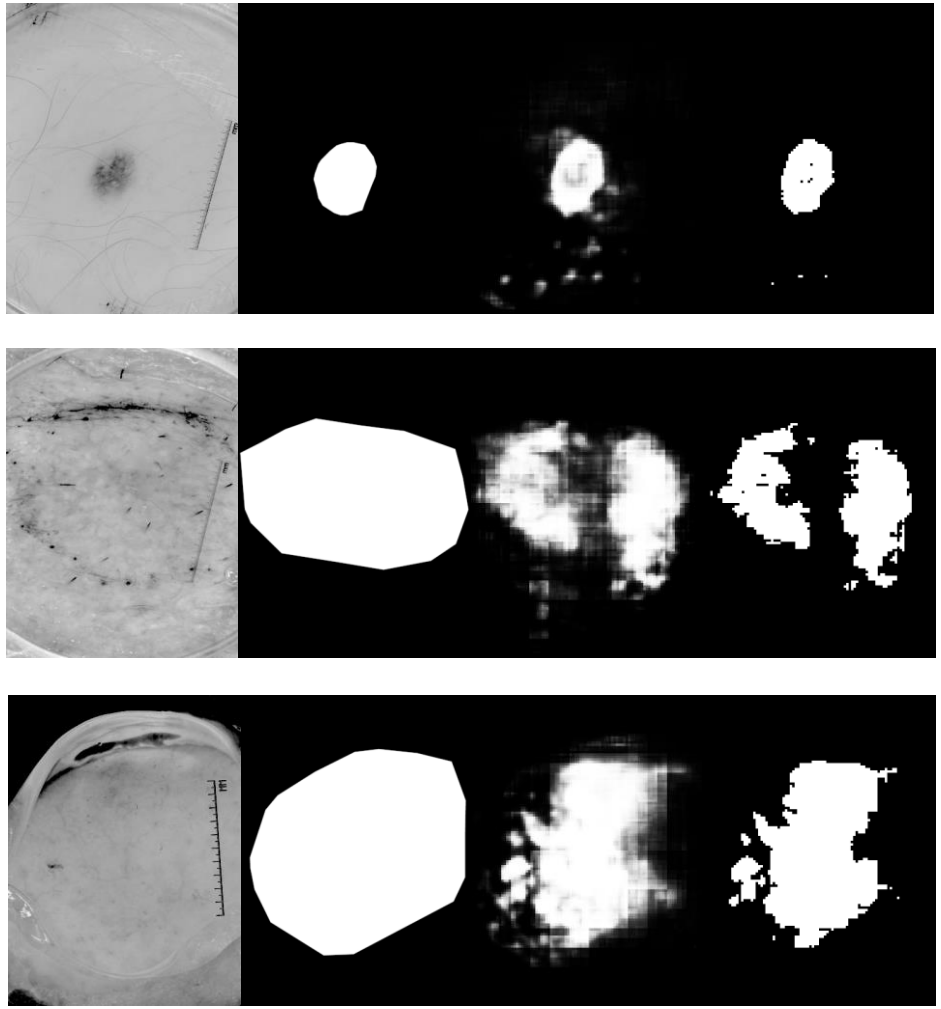
Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.846	0.873	0.899	0.920	0.975	0.957
Test Set	0.782	0.759	0.874	0.889	0.964	0.926

Performance of model without ADAM

Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.885	0.906	0.937	0.924	0.985	0.976
Test Set	0.829	0.813	0.897	0.887	0.984	0.949

Performance of model with ADAM

Experiment Results & Analysis:
Comparison Test 3: add ADAM & not



Test set prediction samples of Non-ADAM model
(image, label, pred, bpred)

Performance is really bad, especially for large-scaled samples!

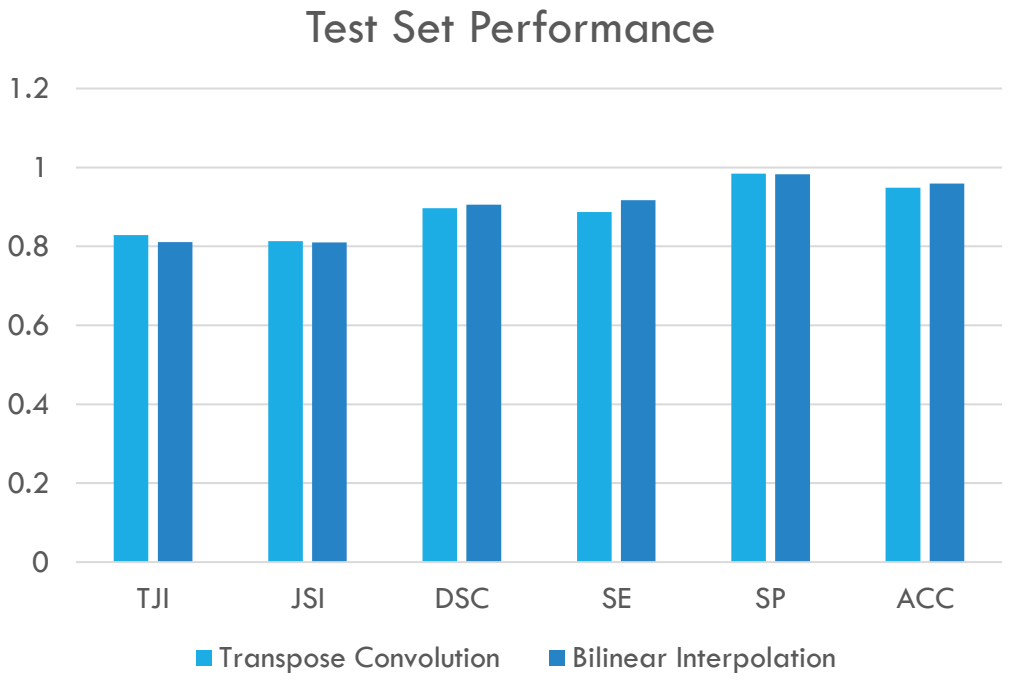
Experiment Results & Analysis:
Comparison Test 4: up-sample by trans conv & bili interpolate

Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.885	0.906	0.937	0.924	0.985	0.976
Test Set	0.829	0.813	0.897	0.887	0.984	0.949

Performance of model with bilinear interpolation up-sampling

Indicators	TJI	JSI	DSC	SE	SP	ACC
Train Set	0.879	0.912	0.944	0.938	0.989	0.983
Test Set	0.811	0.810	0.906	0.917	0.983	0.959

Performance of model with transpose convolution up-sampling



Maybe more trainable parameters makes the up-sampling process more task-oriented.

THANK YOU!

XU Chentianye